

Unique String Demonstration

Title: Longest Substring Without Repeating Characters

Objective/Aim:

To find the longest substring without repeating characters using Java string methods, loops, and Scanner input[cite: 26].

System Requirements :

Hardware

- Standard PC/Laptop[cite: 1]
- Intel Core i3/i5 processor or equivalent[cite: 1]
- Minimum 4 GB / 8 GB RAM[cite: 1]
- Minimum 500 MB free storage[cite: 1]
- Keyboard and Monitor[cite: 1]

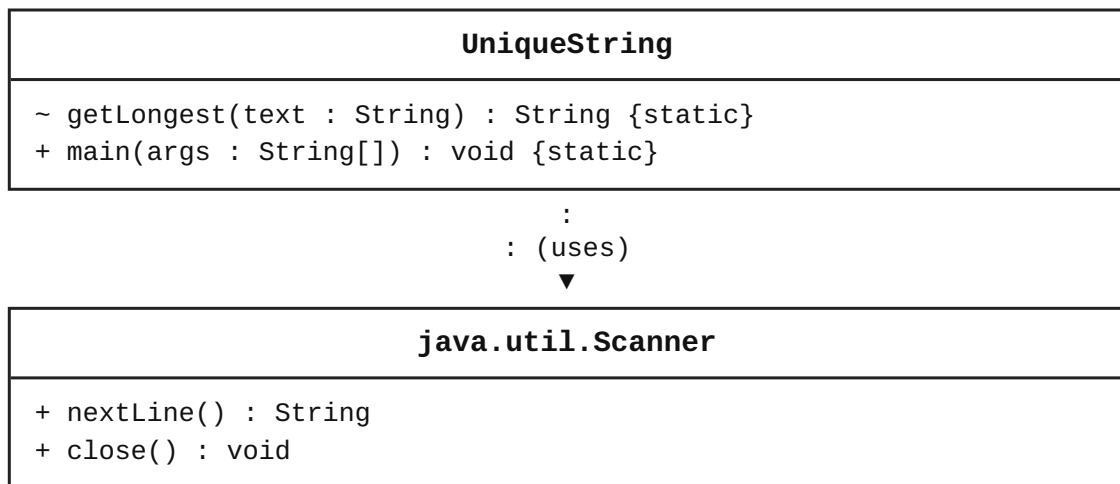
Software

- Operating System: Windows/Linux/macOS[cite: 1]
- Java Development Kit (JDK 17 or JDK 21)[cite: 1]
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code[cite: 1, 26]
- Command Prompt/Terminal[cite: 1]

Algorithm / Logic Description:

1. Read a string using Scanner[cite: 26].
2. Start from each character and create a temporary substring[cite: 26].
3. Add characters while checking for duplicates using indexOf () [cite: 26].
4. Stop when a duplicate character is found[cite: 26].
5. Compare the current substring length with the longest result[cite: 26].
6. Display the longest length and substring[cite: 26].
7. Close the Scanner object.
8. Stop the program.

UML class Diagram



Relationship: The `UniqueString` class uses the `java.util.Scanner` class to accept string input from the terminal[cite: 26].

Source Code:

```
package core_java;

import java.util.Scanner;

public class UniqueString {

    static String getLongest(String text) {

        String answer = "";

        for (int start = 0; start < text.length(); start++) {

            String temp = "";

            for (int end = start; end < text.length(); end++) {

                char letter = text.charAt(end);

                // Check for duplicate character
                if (temp.indexOf(letter) >= 0) {
                    break;
                }

                temp += letter;

                // Compare current substring with previous result
                if (temp.length() >= answer.length()) {
                    answer = temp;
                }
            }
        }
    }
}
```

```

        return answer;
    }

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String text = input.nextLine();

        String result = getLongest(text);

        System.out.println("Longest length: " + result.length());
        System.out.println("Longest substring: " + result);

        input.close();
    }
}

```

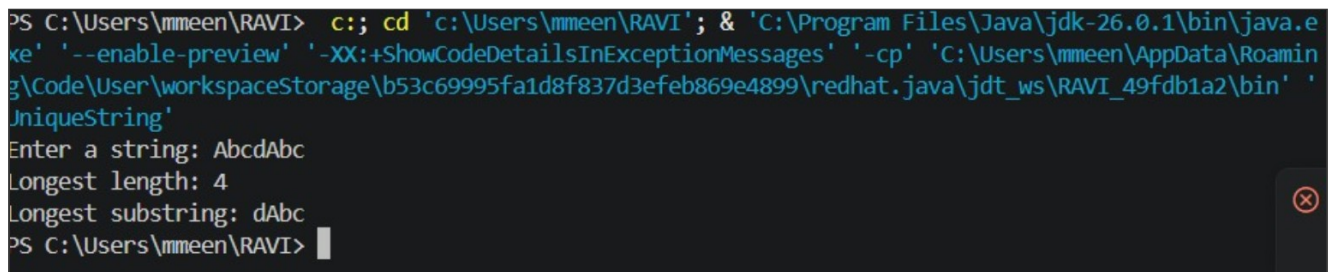
Compilation & Execution Commands:

Save file as: UniqueString.java[cite: 27]

Compilation: javac -d . UniqueString.java (or javac UniqueString.java)[cite: 27]

Execution: java core_java.UniqueString (or java UniqueString)[cite: 27]

Output Screenshot:



```

PS C:\Users\mmeen\RAVI> c::; cd 'c:\Users\mmeen\RAVI'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\mmeen\AppData\Roaming\Code\User\workspaceStorage\b53c69995fa1d8f837d3efeb869e4899\redhat.java\jdt_ws\RAVI_49fdb1a2\bin' 'UniqueString'
Enter a string: AbcdAbc
Longest length: 4
Longest substring: dAbc
PS C:\Users\mmeen\RAVI>

```

Result:

The program was implemented successfully to find the longest substring containing unique characters[cite: 27]. The implementation confirms the proper use of nested loops, condition checking using `indexOf()`, and string manipulation in Java[cite: 26].